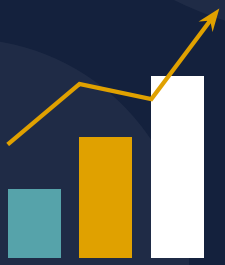
MAWLANA BHASHANI SCIENCE AND TECHNOLOGY UNIVERSITY

Santosh, Tangail-1902, Bangladesh | Department of Information and Communication Technology

SOFTWARE ENGINEERING • PROJECT PROPOSAL

AI-Powered Personal Finance Assistant

Expense Analysis • Budget Planning
Savings Prediction • Financial Chatbot



Version 1.0 – Initial Draft

SUBMITTED BY

MD. MOHIB HASAN
Student ID: **IT24058**
Department of ICT, MBSTU

SUBMITTED TO

Prof. Dr. Ziaur Rahaman
Department of ICT
MBSTU

PROCESS MODEL
Incremental Model with Scrum-style sprints

DURATION
16 weeks (2 Nov 2026 – 21 Feb 2027)

SUBMISSION
October 2026

Table of Contents

1 Introduction **1**
- 1.1 Background . . . 1
- 1.2 Problem Statement . . . 1
- 1.3 Aim and Objectives . . . 1
- 1.4 Scope . . . 2
- 1.5 Significance . . . 2

2 Review of Existing Solutions **2**

3 Requirements Specification **2**
- 3.1 Stakeholders . . . 2
- 3.2 Functional Requirements . . . 2
- 3.3 Non-Functional Requirements . . . 3

4 Proposed System and Technologies **3**
- 4.1 System Architecture . . . 3
- 4.2 AI Components . . . 3

5 Software Process Model **4**

6 Project Schedule, Milestones and Deliverables **4**
- 6.1 Work Breakdown and Timeline . . . 4
- 6.2 Gantt Chart . . . 5
- 6.3 Milestones . . . 5
- 6.4 Deliverables . . . 6

7 Feasibility, Resources and Cost **6**

8 Risk Management **7**

9 Quality Assurance and Testing **7**

10 System Models (UML) **8**
- 10.1 Data Flow Diagram (DFD) . . . 8
- 10.2 Use Case Diagram . . . 9
- 10.3 Activity Diagram . . . 10
- 10.4 Entity Relationship Diagram (ERD) . . . 11
- 10.5 Class Diagram . . . 12
- 10.6 Object Diagram . . . 12

11 Prototype Interface Implementation **13**
- 11.1 Source Code . . . 14

12 Expected Outcomes and Success Criteria **17**

13 Future Work **17**

14 Conclusion **17**

Abstract

Many students and young professionals in Bangladesh track money in notebooks, memory, or mobile-wallet histories (bKash, Nagad, Rocket) and rarely analyse where it goes. This proposal presents the *AI-Powered Personal Finance Assistant (AI-PFA)*, a software system that (i) records income and expenses, (ii) analyses spending by category and detects unusual transactions, (iii) helps the user create and monitor monthly budgets, (iv) predicts future savings using machine learning, and (v) answers financial questions through a conversational chatbot. The proposal describes the problem, objectives, scope, requirements, process model, schedule, milestones, deliverables, risks, test plan and a complete set of UML models (DFD, use case, activity, ER, class and object diagrams). A working prototype interface built from the classes of the class diagram is included in Section 11.

16	7	10	6
WEEKS	MILESTONES	DELIVERABLES	UML DIAGRAMS

1 Introduction

1.1 Background

Personal financial literacy is a key life skill, yet most people lack simple tools that turn raw transactions into advice. Existing apps are either manual trackers with no intelligence or complex products designed for other markets. Advances in data analysis, regression models and natural-language interfaces make it feasible to build a low-cost assistant that gives personalised, understandable guidance.

1.2 Problem Statement

Users (1) do not know how much they spend on each category, (2) overspend because budgets are not enforced or monitored, (3) cannot estimate how much they will save in the coming months, and (4) have no easy way to ask financial questions in plain language. A system that automates these tasks is needed.

1.3 Aim and Objectives

Aim

To design and develop an intelligent, easy-to-use personal finance assistant that turns everyday transactions into clear insight, budgets and savings predictions.

Objectives:

1. Provide secure registration, login and management of income/expense transactions.
2. Automatically categorise transactions and visualise spending patterns.
3. Allow budget creation with real-time warnings at 80% and 100% of the limit.
4. Predict next-month savings with a regression model (target MAE $<$ 10% of mean monthly savings).
5. Provide a chatbot that answers questions such as "Where did I spend most this month?"
6. Deliver complete documentation, UML models, tests and a working prototype.

1.4 Scope

In scope: single-user accounts, manual and CSV transaction entry, categories, budgets, savings goals, analytics dashboard, savings forecast, intent-based chatbot, reports (PDF/CSV), basic admin panel.
Out of scope (future work): direct bank/bKash API integration, investment advice, multi-currency, shared family accounts, native mobile app.

1.5 Significance

The project demonstrates a full software-engineering life cycle and combines data analytics with AI in a socially useful application, helping users build saving habits and avoid debt.

2 Review of Existing Solutions

Tool	Strength	Limitation	AI-PFA advantage
Paper/Excel tracking	Free, flexible	Manual, no insight	Automatic analysis
Mint/YNAB-style apps	Rich features	Paid, foreign banks	Local context (BDT)
Wallet transaction history	Accurate records	No budgeting/prediction	Budgets + forecasting
Generic chatbots	Conversational	Do not know user's data	Personalised answers

Table 1. *Comparison with existing approaches*

3 Requirements Specification

3.1 Stakeholders

End users (students, employees, families); the project supervisor (Prof. Dr. Ziaur Rahaman); the developer (MD. Mohib Hasan); and the system administrator.

3.2 Functional Requirements

ID	Requirement	Priority
FR1	The system shall allow a user to register, log in, log out and reset a password.	High
FR2	The system shall allow the user to add, edit, delete and search income/expense transactions.	High
FR3	The system shall support user-defined and default categories.	High
FR4	The system shall import transactions from a CSV file.	Medium
FR5	The system shall show spending by category and by month (charts).	High
FR6	The system shall detect abnormal expenses (z-score / Isolation Forest).	Medium
FR7	The system shall let the user create monthly category budgets and warn at 80% and 100%.	High
FR8	The system shall let the user set savings goals and track progress.	Medium
FR9	The system shall predict the next-month savings and goal completion date.	High

ID	Requirement	Priority
FR10	The system shall provide a chatbot answering spending, budget and savings queries.	High
FR11	The system shall export reports as PDF/CSV.	Low
FR12	The administrator shall manage users and default categories.	Medium

3.3 Non-Functional Requirements

- **Security:** passwords hashed (bcrypt), HTTPS, role-based access, per-user data isolation.
- **Performance:** dashboard loads in < 3 s for 10,000 transactions; chatbot reply in < 2 s.
- **Usability:** a new user can add a transaction in < 30 s without training.
- **Reliability:** 99% availability in testing; daily database backup.
- **Portability:** runs on Windows/Linux; responsive browser interface.
- **Maintainability:** modular, object-oriented design with $> 70\%$ unit-test coverage.
- **Privacy:** data is used only for the user's own analysis; the user can delete all data.

4 Proposed System and Technologies

4.1 System Architecture

A three-tier architecture is used: **Presentation** (web/desktop UI), **Application** (REST API with business classes and AI modules) and **Data** (relational database).

Layer	Technology
Language	Python 3.11
Backend / API	Flask or FastAPI
Database	SQLite (development), PostgreSQL (deployment)
Data analysis	pandas, NumPy
Machine learning	scikit-learn (Linear/Ridge Regression, Random Forest, Isolation Forest)
NLP / Chatbot	Intent classification (TF-IDF + Logistic Regression); optional LLM API
Frontend	HTML/CSS/JavaScript with Chart.js; Tkinter prototype
Version control / CI	Git, GitHub, pytest
Documentation	LaTeX, draw.io / TikZ

Table 3. *Technology stack*

4.2 AI Components

Savings prediction. Monthly savings $s_t = I_t - E_t$ (income minus expense). A linear trend model is fitted by least squares:

$$\hat{s}_{t+1} = \beta_0 + \beta_1 (t+1), \qquad \beta_1 = \frac{\sum_t (t-\bar{t})(s_t - \bar{s})}{\sum_t (t-\bar{t})^2}, \quad \beta_0 = \bar{s} - \beta_1 \bar{t}.$$

Richer features (category totals, festival months, rolling averages) and Random Forest/ARIMA are compared using MAE, RMSE and R^2 on a time-ordered train/test split.

Anomaly detection. A transaction x in a category is flagged if $z = (x - \mu)/\sigma > 2$; Isolation Forest is evaluated as an alternative.

Auto-categorisation. The transaction note (e.g. "rickshaw to campus") is classified into a category using TF-IDF features and Logistic Regression.

Chatbot. Intents (spending summary, budget status, forecast, advice) are detected from the user's message and answered from the user's own data; a 50/30/20 rule engine provides saving tips.

5 Software Process Model

The **Incremental Model** with **Scrum-style** 3-week sprints is chosen. Core requirements are clear, but the AI modules benefit from early feedback and tuning. Each increment delivers a usable, tested part of the system.

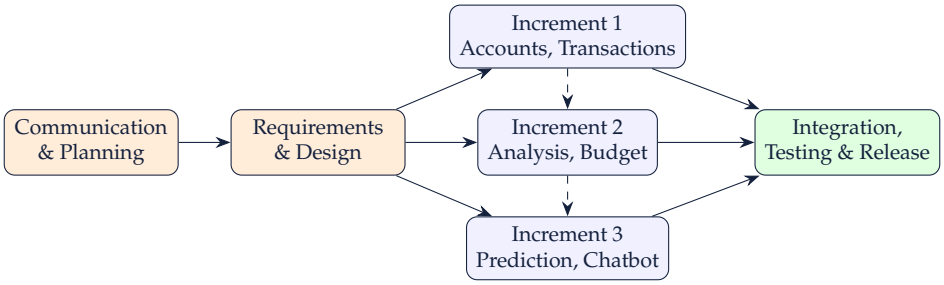


Figure 1. *Incremental development plan (each increment: design → code → test → review)*

6 Project Schedule, Milestones and Deliverables

6.1 Work Breakdown and Timeline

The project lasts 16 weeks (2 Nov 2026 – 21 Feb 2027); dates may shift with the academic calendar.

Phase	Weeks	Main activities	Dates
Planning	1	Topic study, feasibility, proposal, risk plan	2–8 Nov
Requirements	1–2	Interviews/survey, SRS, use cases	2–15 Nov
Design	3–4	UML models, database schema, UI wireframes	16–29 Nov
Sprint 1	5–7	Authentication, transaction CRUD, categories	30 Nov–20 Dec
Sprint 2	8–10	Analytics dashboard, anomalies, budgets, goals	21 Dec–10 Jan
Sprint 3	11–13	Savings prediction, auto-categorisation, chatbot	11 Jan–31 Jan
Testing	14–15	Integration, system, usability tests, bug fixing	1–14 Feb
Release	15–16	Deployment, documentation, final report, demo	8–21 Feb

Table 4. *Phase plan*

6.2 Gantt Chart

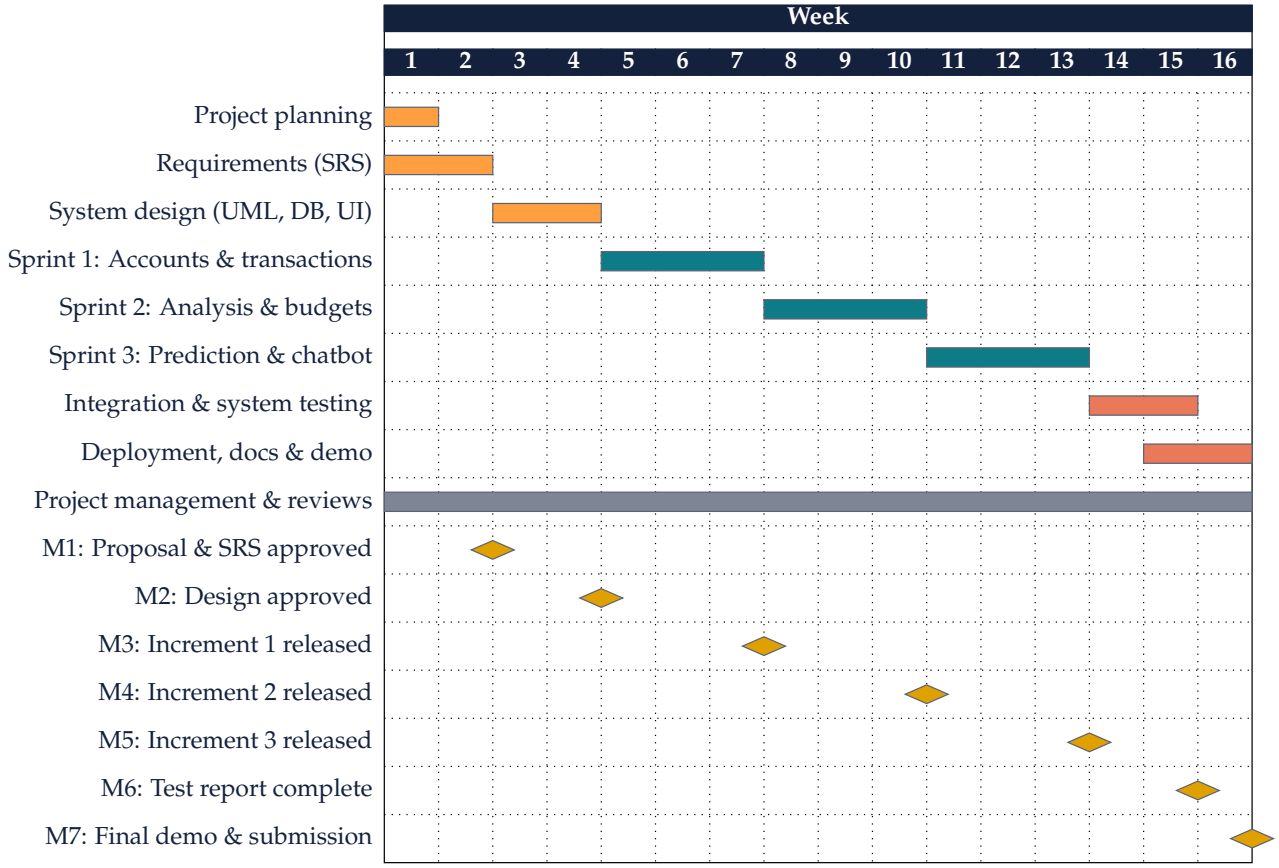


Figure 2. *Gantt chart of the 16-week project plan (orange: planning & design, teal: development sprints, red: testing & release, gold: milestones)*

6.3 Milestones

ID	Week	Milestone / exit criterion	Date
M1	2	Proposal and SRS reviewed and approved by supervisor	15 Nov
M2	4	All UML diagrams, ER schema and UI wireframes approved	29 Nov
M3	7	Users can register, log in and manage transactions (FR1–FR4)	20 Dec
M4	10	Dashboard, anomaly detection, budgets and goals work (FR5–FR8)	10 Jan
M5	13	Prediction and chatbot meet accuracy targets (FR9–FR10)	31 Jan
M6	15	All test cases executed; no open critical defects	14 Feb
M7	16	Final demo, report and source code submitted	21 Feb

Table 5. *Project milestones*

6.4 Deliverables

ID	Deliverable	Description	Due
D1	Project proposal	This document with schedule and UML	Wk 1
D2	SRS document	Functional/non-functional requirements, use cases	Wk 2
D3	Design document	DFD, UML, ER, database schema, wireframes	Wk 4
D4	Increment 1	Accounts and transactions module + unit tests	Wk 7
D5	Increment 2	Analytics, budgets, goals module + unit tests	Wk 10
D6	Increment 3	ML prediction model, chatbot + evaluation report	Wk 13
D7	Test report	Test plan, cases, results, defect log, coverage	Wk 15
D8	Final system	Deployed application and source code (GitHub)	Wk 16
D9	User manual	Installation and user guide	Wk 16
D10	Final report & slides	Report, presentation and demo video	Wk 16

Table 6. *Project deliverables*

7 Feasibility, Resources and Cost

Technical: all tools are open source and well documented. **Economic:** the cost is minimal (below). **Operational:** the interface is simple and in the user's currency (BDT). **Schedule:** the plan fits 16 weeks with a one-week buffer built into testing.

Item	Qty	Est. cost (BDT)
Developer effort (self, ≈10 h/week × 16 weeks)	160 h	–
Personal laptop (existing)	1	0
Open-source software (Python, Git, LaTeX, DB)	–	0
Internet and data	4 months	2,000
Optional cloud hosting / domain	4 months	2,500
Printing and binding of final report	–	800
Total		**5,300**

Table 7. *Estimated budget (approximate)*

8 Risk Management

Risk	Description	Prob.	Impact	Mitigation
R1	Not enough real data to train the model	High	High	Use synthetic data and public datasets; collect consented sample data
R2	Schedule slip due to exams	Med	High	Time buffer; prioritise by MoSCoW; keep increments small
R3	Model accuracy below target	Med	Med	Compare several models; fall back to the simple trend model
R4	Security/privacy breach	Low	High	Hashing, input validation, HTTPS, no sharing of data
R5	Scope creep	Med	Med	Fixed scope baseline; new ideas go to the backlog
R6	Data loss / hardware failure	Low	High	Git, daily backups

Table 8. *Risk register*

9 Quality Assurance and Testing

- **Unit tests** (pytest) for every class; **integration tests** for API + database; **system tests** against FR1–FR12; **usability test** with at least 10 users (SUS questionnaire, target score ≥ 70).
- **ML evaluation:** MAE, RMSE, R^2 for prediction; accuracy/F1 for categorisation and intents.
- **Reviews:** weekly supervisor meeting, sprint review and retrospective, code review checklist.
- **Configuration management:** Git branches (main/dev/feature), tagged release per increment.

10 System Models (UML)

10.1 Data Flow Diagram (DFD)

Level 0 – Context diagram.

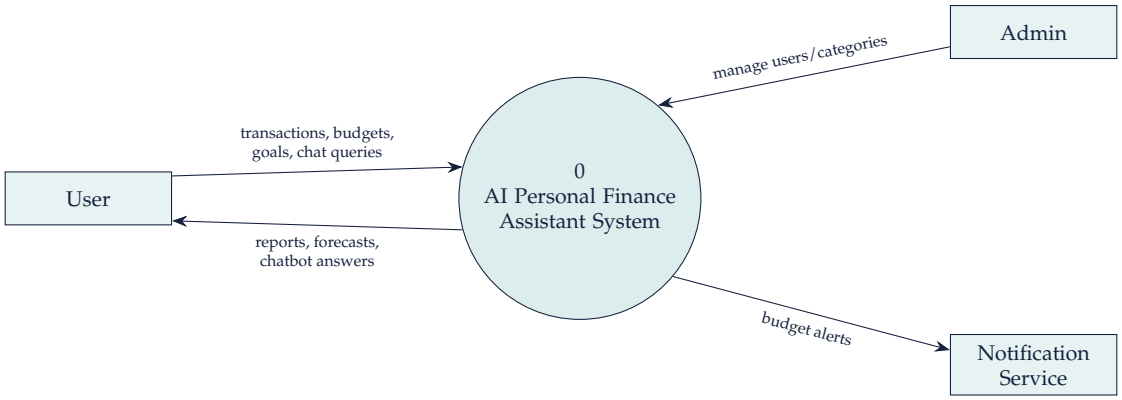


Figure 3. *DFD Level 0 (context diagram)*

Level 1.

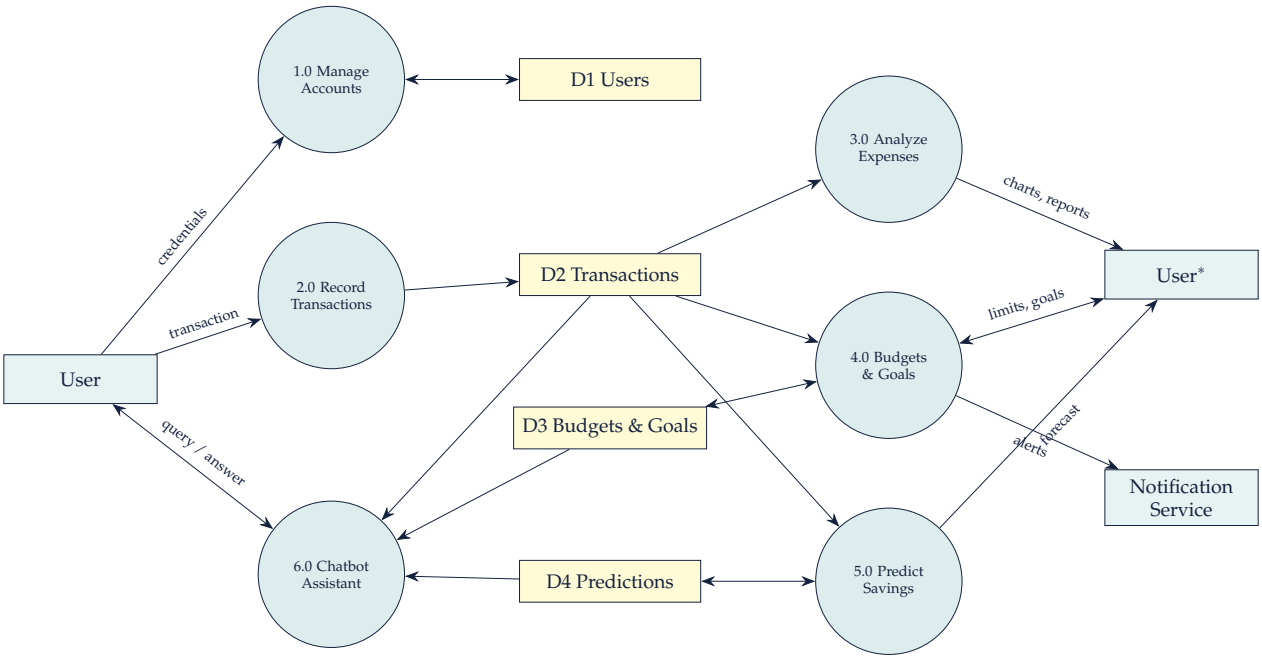


Figure 4. *DFD Level 1 (User* is the same external entity repeated to avoid crossing lines)*

10.2 Use Case Diagram

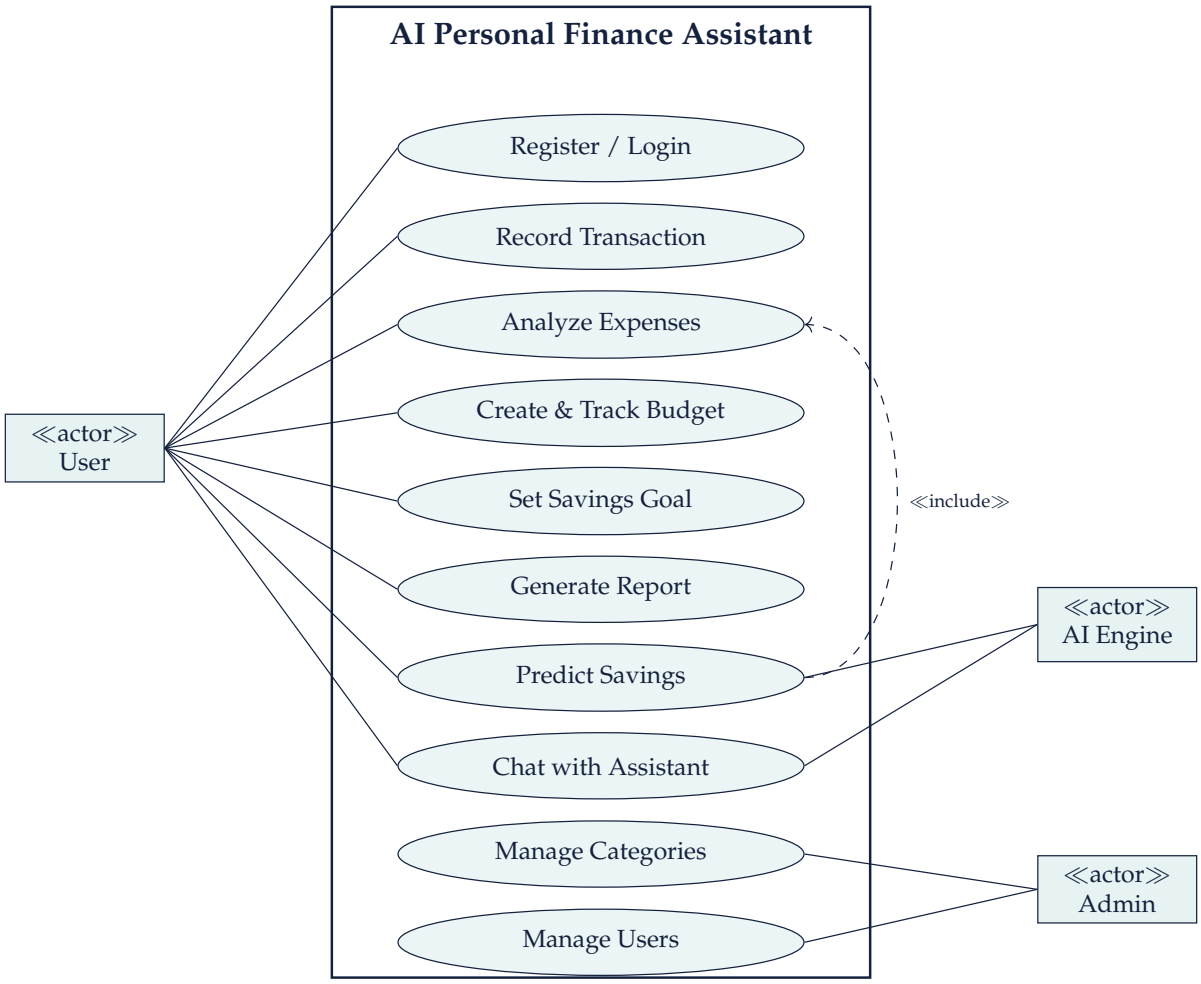


Figure 5. *Use case diagram*

Use case: Record Transaction	
Actor	User
Pre-condition	User is logged in
Main flow	1) User selects "Add transaction". 2) Enters amount, category, date and note. 3) System validates data. 4) System saves the transaction. 5) System updates analysis and budget status.
Alternative flow	3a) Invalid data: system shows an error and asks the user to correct it.
Post-condition	Transaction stored; dashboard and predictions are refreshed

Table 9. *Use case specification*

10.3 Activity Diagram

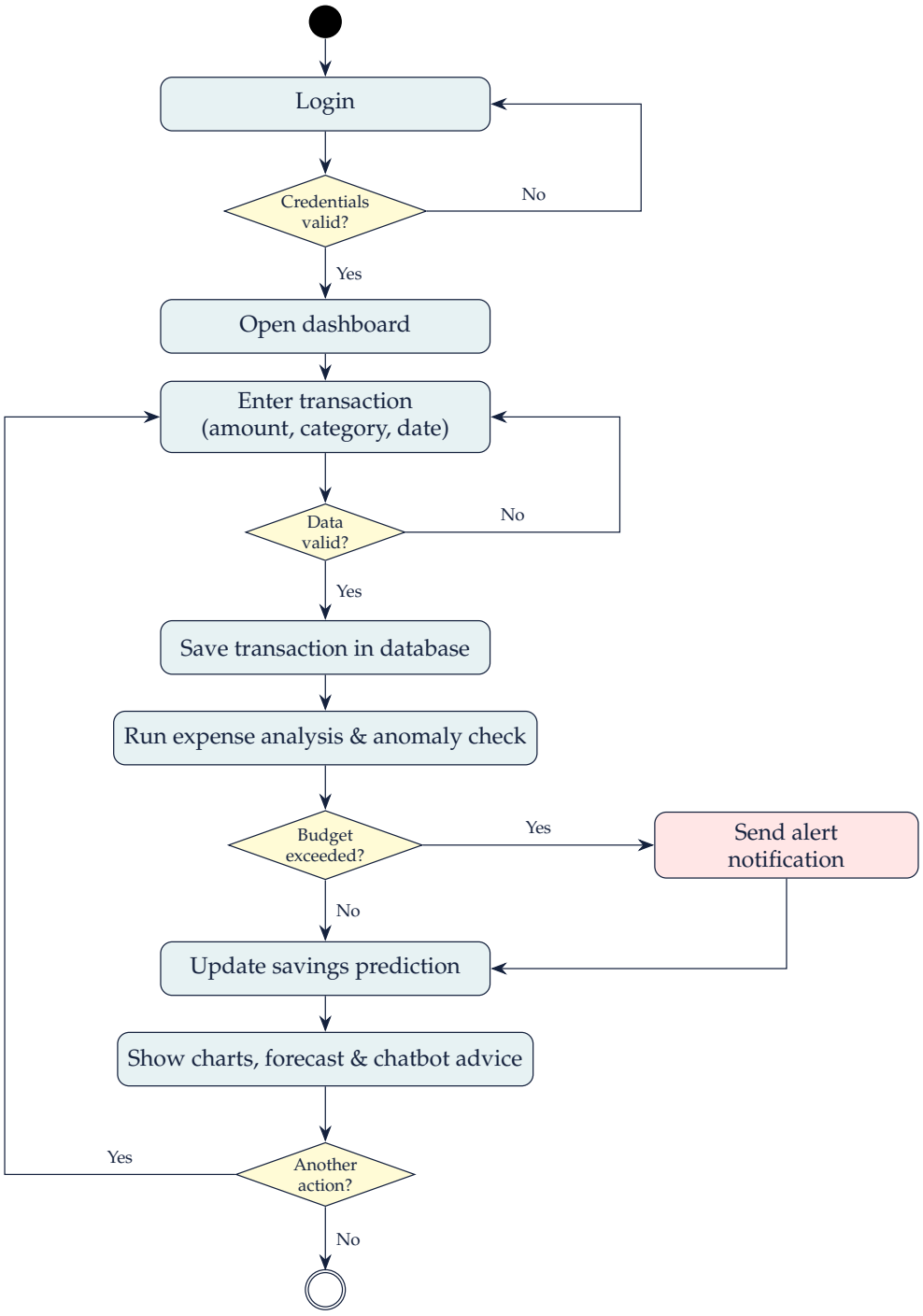


Figure 6. *Activity diagram: recording a transaction and receiving analysis*

10.4 Entity Relationship Diagram (ERD)

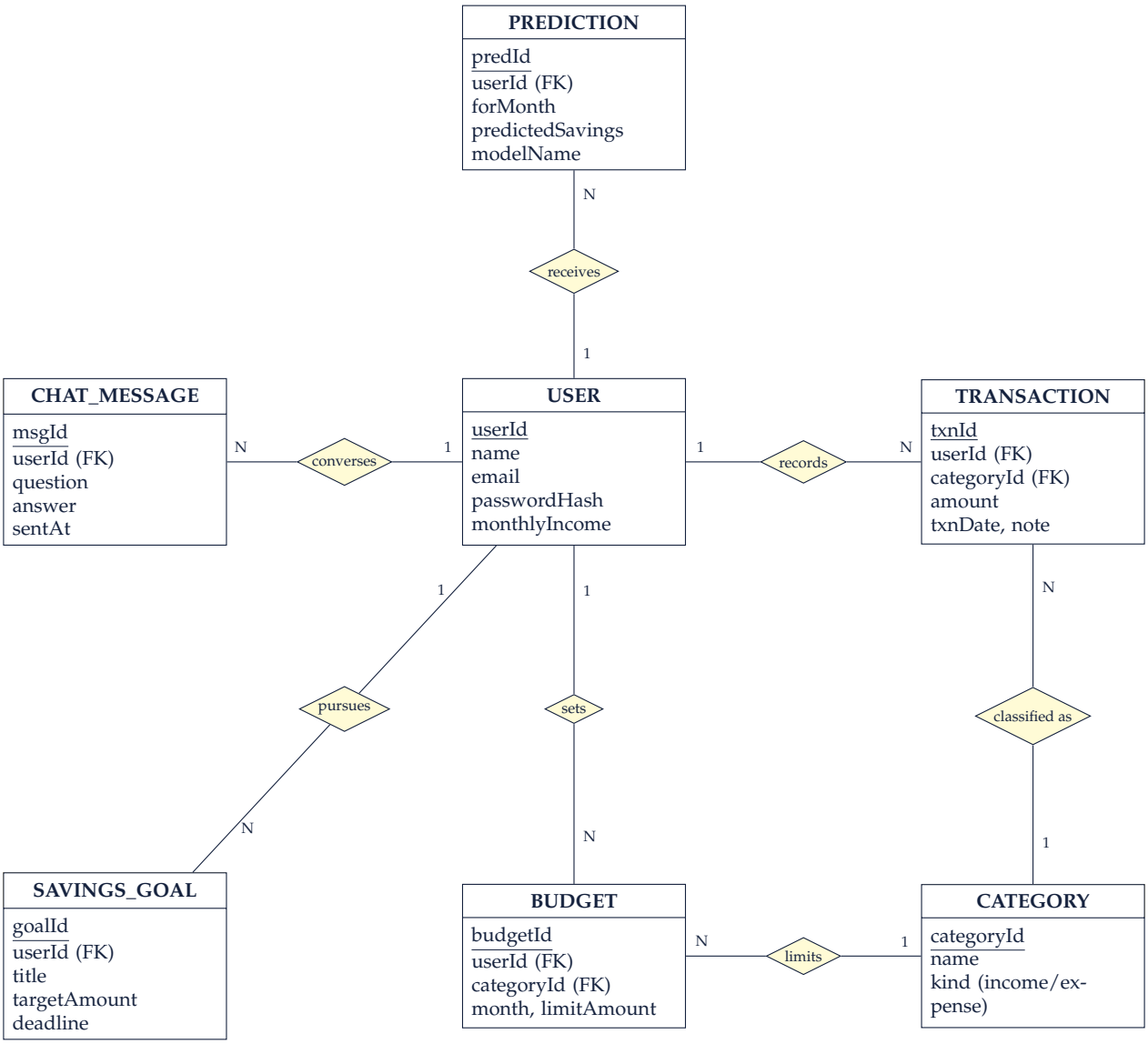


Figure 7. *Entity–relationship diagram (Chen notation; PK underlined, FK marked)*

10.5 Class Diagram

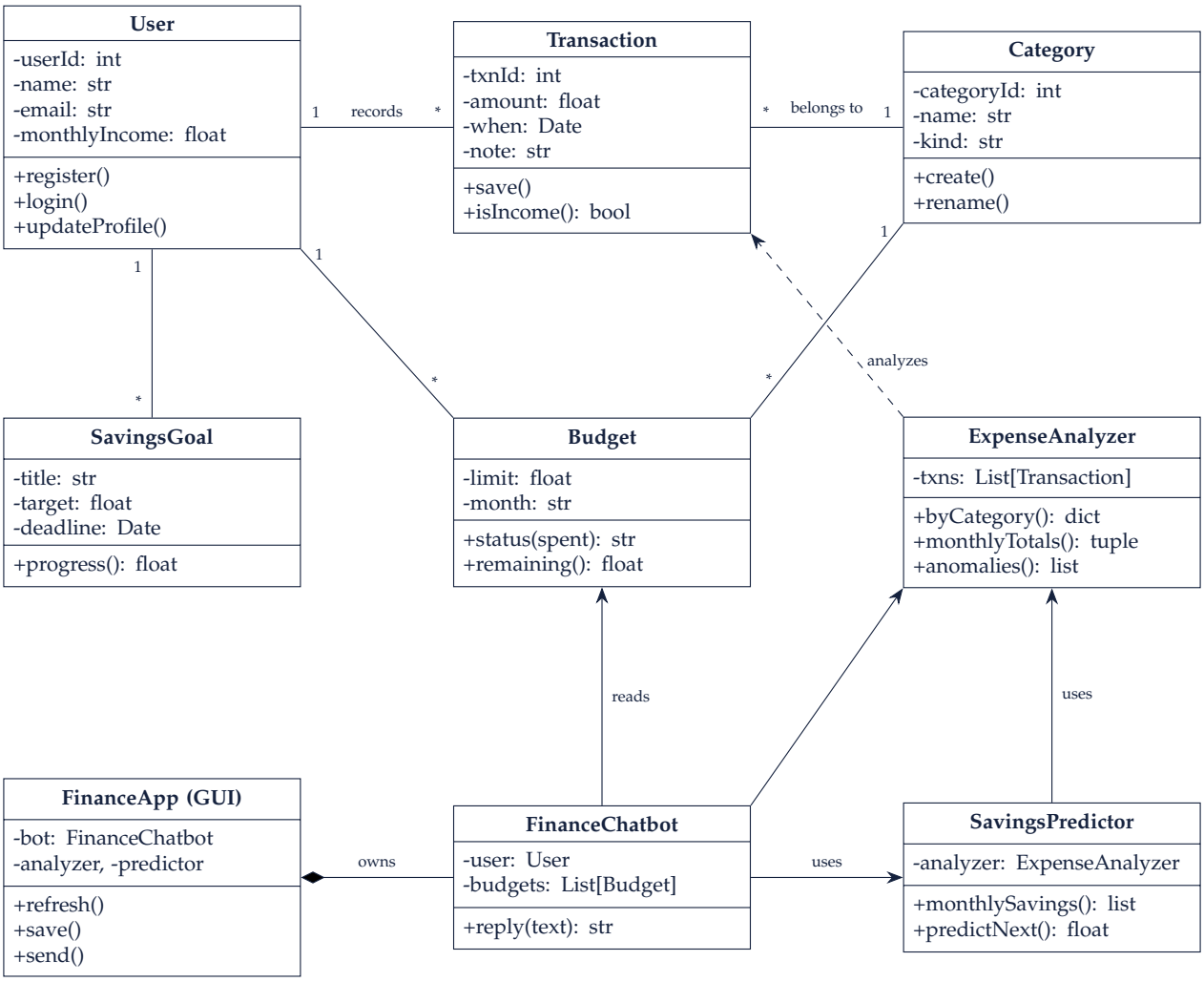


Figure 8. *Class diagram*

10.6 Object Diagram

Snapshot of the system at run time (October 2026):

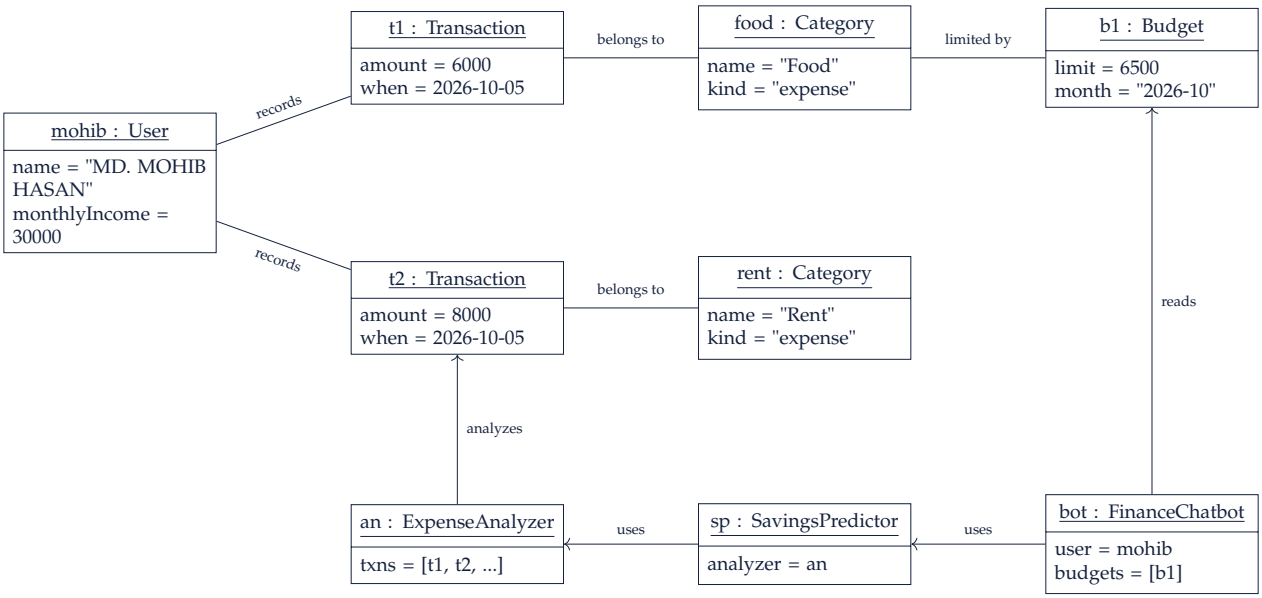


Figure 9. *Object diagram (instances of the classes in the class diagram above)*

11 Prototype Interface Implementation

A simple Tkinter interface (`finance_assistant.py`) was implemented from the class diagram. It uses **eight classes** from the class diagram: `User`, `Category`, `Transaction`, `Budget`, `ExpenseAnalyzer`, `SavingsPredictor`, `FinanceChatbot` and the GUI class `FinanceApp` (`SavingsGoal` is planned for Sprint 2). The program has three tabs.

Tab	Function and classes used
Dashboard	Shows spending by category, predicted savings, budget status and anomaly count (`ExpenseAnalyzer`, `SavingsPredictor`, `Budget`)
Add Transaction	Form to enter amount, category and note; creates a `Transaction` linked to a `Category`
Chatbot	Text box answering "Where did I spend most?", "Show my budget", "Predict next month savings" (`FinanceChatbot`)

Table 10. *Interface tabs*

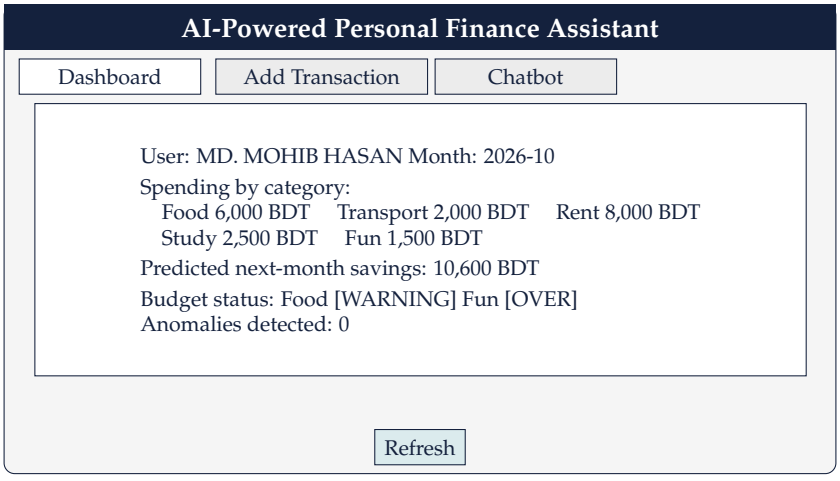


Figure 10. *Wireframe of the Dashboard tab of the prototype*

Console demo output (`python finance_assistant.py –demo`), used to verify the business classes:

```
Category spend: {'Food': 6000.0, 'Transport': 2000.0, 'Rent': 8000.0, 'Study': 2500.0, 'Fun':
    1500.0}
Predicted savings: 10600.0
> Where did I spend most?   You spent 20,000 BDT this month; highest: Rent (8,000).
> Show my budget            Food: 6,000/6,500 [WARNING]   Fun: 1,500/1,200 [OVER]
> Predict next month savings  Predicted savings next month: 10,600 BDT.
```

11.1 Source Code

```python
"""AI-Powered Personal Finance Assistant -- prototype interface.
Author: MD. MOHIB HASAN (IT24058), Dept. of ICT, MBSTU.
Classes (mirror the UML class diagram): User, Category, Transaction, Budget,
ExpenseAnalyzer, SavingsPredictor, FinanceChatbot, FinanceApp (GUI).
Run:  python finance_assistant.py        (Tkinter GUI)
      python finance_assistant.py --demo (console demo, no GUI)
"""
from dataclasses import dataclass, field
from datetime import date, timedelta
from collections import defaultdict
import sys


@dataclass
class User:
    user_id: int
    name: str
    email: str
    monthly_income: float = 0.0


@dataclass
class Category:
    category_id: int
    name: str
    kind: str = "expense"          # "expense" or "income"


@dataclass
class Transaction:
    txn_id: int
    amount: float
    category: Category
    when: date
    note: str = ""

    @property
    def is_income(self):
        return self.category.kind == "income"


@dataclass
class Budget:
    category: Category
    limit: float
    month: str                      # "YYYY-MM"

    def status(self, spent):
        pct = 100 * spent / self.limit if self.limit else 0
        return "OVER" if pct > 100 else "WARNING" if pct >= 80 else "OK"


class ExpenseAnalyzer:
    def __init__(self, txns):
        self.txns = txns

    def by_category(self, month=None):
        out = defaultdict(float)
        for t in self.txns:
            if not t.is_income and (month is None or t.when.strftime("%Y-%m") == month):
                out[t.category.name] += t.amount
```

```
        return dict(out)

    def monthly_totals(self):
        inc, exp = defaultdict(float), defaultdict(float)
        for t in self.txns:
            (inc if t.is_income else exp)[t.when.strftime("%Y-%m")] += t.amount
        return dict(inc), dict(exp)

    def anomalies(self, z=2.0):
        """Flag expenses far above the category mean (z-score rule)."""
        groups = defaultdict(list)
        for t in self.txns:
            if not t.is_income:
                groups[t.category.name].append(t)
        flagged = []
        for items in groups.values():
            vals = [t.amount for t in items]
            if len(vals) < 4:
                continue
            m = sum(vals) / len(vals)
            sd = (sum((v - m) ** 2 for v in vals) / len(vals)) ** 0.5
            flagged += [t for t in items if sd and (t.amount - m) / sd > z]
        return flagged


class SavingsPredictor:
    """Simple linear regression (least squares) on monthly savings."""
    def __init__(self, analyzer):
        self.analyzer = analyzer

    def monthly_savings(self):
        inc, exp = self.analyzer.monthly_totals()
        return [inc.get(m, 0) - exp.get(m, 0) for m in sorted(set(inc) | set(exp))]

    def predict_next(self, k=1):
        y = self.monthly_savings()
        n = len(y)
        if n < 2:
            return y[-1] if y else 0.0
        xs = range(n)
        mx, my = (n - 1) / 2, sum(y) / n
        den = sum((x - mx) ** 2 for x in xs)
        slope = sum((x - mx) * (v - my) for x, v in zip(xs, y)) / den
        return my + slope * ((n - 1 + k) - mx)


class FinanceChatbot:
    """Rule/intent-based assistant (can be swapped for an LLM API)."""
    def __init__(self, user, analyzer, predictor, budgets):
        self.user, self.an, self.pred, self.budgets = user, analyzer, predictor, budgets

    def reply(self, text):
        q = text.lower()
        month = date.today().strftime("%Y-%m")
        if any(w in q for w in ("predict", "forecast", "next month", "saving")):
            return f"Predicted savings next month: {self.pred.predict_next():,.0f} BDT."
        if any(w in q for w in ("budget", "limit")):
            spent = self.an.by_category(month)
            lines = [f"{b.category.name}: {spent.get(b.category.name, 0):,.0f}/{b.limit:,.0f} "
                     f"[{b.status(spent.get(b.category.name, 0))}]" for b in self.budgets]
            return "\n".join(lines) or "No budgets set yet."
        if any(w in q for w in ("spend", "expense", "where", "top")):
            d = self.an.by_category(month)
            if not d:
                return "No expenses recorded this month."
            top = max(d, key=d.get)
            return f"You spent {sum(d.values()):,.0f} BDT this month; highest: {top} ({d[top
                ]:,.0f})."
        if any(w in q for w in ("tip", "advice", "help")):
            return "Tip: follow the 50/30/20 rule -- needs / wants / savings."
        return "Ask me about spending, budget, or savings prediction."


def sample_data():
    user = User(1, "MD. MOHIB HASAN", "mohib@example.com", 30000)
```

```
    inc = Category(0, "Salary", "income")
    cats = {n: Category(i + 1, n) for i, n in enumerate(["Food", "Transport", "Rent", "Study", "Fun"])}
    txns, tid = [], 1
    base = {"Food": 6000, "Transport": 2000, "Rent": 8000, "Study": 2500, "Fun": 1500}
    today = date.today().replace(day=1)
    for back in range(5, -1, -1):
        d = (today - timedelta(days=30 * back)).replace(day=5)
        txns.append(Transaction(tid, 30000, inc, d, "Monthly income")); tid += 1
        for n, c in cats.items():
            txns.append(Transaction(tid, base[n] + back * 120, c, d, n)); tid += 1
    budgets = [Budget(cats["Food"], 6500, today.strftime("%Y-%m")),
               Budget(cats["Fun"], 1200, today.strftime("%Y-%m"))]
    return user, cats, inc, txns, budgets


def run_gui():
    import tkinter as tk
    from tkinter import ttk, messagebox

    class FinanceApp(tk.Tk):
        def __init__(self):
            super().__init__()
            self.title("AI-Powered Personal Finance Assistant")
            self.geometry("760x520")
            self.user, self.cats, self.inc, self.txns, self.budgets = sample_data()
            self.an = ExpenseAnalyzer(self.txns)
            self.pred = SavingsPredictor(self.an)
            self.bot = FinanceChatbot(self.user, self.an, self.pred, self.budgets)
            nb = ttk.Notebook(self); nb.pack(fill="both", expand=True)
            self._tab_dashboard(nb); self._tab_add(nb); self._tab_chat(nb)

        def _tab_dashboard(self, nb):
            f = ttk.Frame(nb); nb.add(f, text="Dashboard")
            self.summary = tk.Text(f, height=22, state="disabled"); self.summary.pack(fill="both", expand=True, padx=8, pady=8)
            ttk.Button(f, text="Refresh", command=self.refresh).pack(pady=4)
            self.refresh()

        def refresh(self):
            m = date.today().strftime("%Y-%m")
            lines = [f"User: {self.user.name}", f"Month: {m}", "", "Spending by category:"]
            for k, v in self.an.by_category(m).items():
                lines.append(f"  {k:<10}{v:>10,.0f} BDT")
            lines += ["", f"Predicted next-month savings: {self.pred.predict_next():,.0f} BDT", "", "Budget status:"]
            sp = self.an.by_category(m)
            for b in self.budgets:
                lines.append(f"  {b.category.name:<10}{b.status(sp.get(b.category.name, 0))}")
            an = self.an.anomalies()
            lines += ["", f"Anomalies detected: {len(an)}"]
            self.summary.config(state="normal"); self.summary.delete("1.0", "end")
            self.summary.insert("end", "\n".join(lines)); self.summary.config(state="disabled")

        def _tab_add(self, nb):
            f = ttk.Frame(nb); nb.add(f, text="Add Transaction")
            ttk.Label(f, text="Amount").grid(row=0, column=0, padx=8, pady=8)
            self.amount = ttk.Entry(f); self.amount.grid(row=0, column=1)
            ttk.Label(f, text="Category").grid(row=1, column=0)
            self.cat = ttk.Combobox(f, values=list(self.cats), state="readonly"); self.cat.current(0); self.cat.grid(row=1, column=1)
            ttk.Label(f, text="Note").grid(row=2, column=0, pady=8)
            self.note = ttk.Entry(f); self.note.grid(row=2, column=1)
            ttk.Button(f, text="Save", command=self.save).grid(row=3, column=1, pady=8)

        def save(self):
            try:
                amt = float(self.amount.get())
            except ValueError:
                return messagebox.showerror("Error", "Enter a valid amount")
            self.txns.append(Transaction(len(self.txns) + 1, amt, self.cats[self.cat.get()], date.today(), self.note.get()))
            self.refresh(); messagebox.showinfo("Saved", "Transaction added")

        def _tab_chat(self, nb):
```

```
            f = ttk.Frame(nb); nb.add(f, text="Chatbot")
            self.log = tk.Text(f, height=20, state="disabled"); self.log.pack(fill="both",
                expand=True, padx=8, pady=8)
            self.inp = ttk.Entry(f); self.inp.pack(fill="x", padx=8); self.inp.bind("<Return>",
                lambda e: self.send())
            ttk.Button(f, text="Send", command=self.send).pack(pady=4)

        def send(self):
            q = self.inp.get().strip()
            if not q: return
            self.inp.delete(0, "end")
            self.log.config(state="normal")
            self.log.insert("end", f"You: {q}\nBot: {self.bot.reply(q)}\n\n")
            self.log.config(state="disabled")

    FinanceApp().mainloop()


def run_demo():
    user, cats, inc, txns, budgets = sample_data()
    an = ExpenseAnalyzer(txns); pr = SavingsPredictor(an); bot = FinanceChatbot(user, an, pr,
        budgets)
    print("Category spend:", an.by_category(date.today().strftime("%Y-%m")))
    print("Predicted savings:", round(pr.predict_next(), 2))
    for q in ("Where did I spend most?", "Show my budget", "Predict next month savings"):
        print(f"> {q}\n{bot.reply(q)}")


if __name__ == "__main__":
    run_demo() if "--demo" in sys.argv else run_gui()
```

12 Expected Outcomes and Success Criteria

Success criteria

- A working AI-PFA covering all high-priority requirements (FR1–FR3, FR5, FR7, FR9, FR10).
- Prediction error (MAE) below 10% of mean monthly savings on the test data.
- Chatbot intent accuracy $\geq$ 85% on the test set.
- SUS usability score $\geq$ 70; test coverage $\geq$ 70%.
- Complete documentation: SRS, design, test report, user manual and final report.

13 Future Work

Direct integration with bKash/Nagad/bank statements, OCR of receipts, Bangla-language chatbot, mobile application, and recommendation of saving plans using reinforcement learning.

14 Conclusion

AI-PFA addresses a real everyday problem with a feasible, well-planned software project. The incremental process, detailed schedule, risk plan, UML design and working prototype show that the project is ready to proceed to the requirements and design phases, subject to the supervisor's approval.

References

[1] I. Sommerville, *Software Engineering*, 10th ed., Pearson, 2016.

[2] R. S. Pressman and B. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed., McGraw-Hill, 2020.

[3] M. Fowler, *UML Distilled*, 3rd ed., Addison-Wesley, 2003.

[4] P. P. Chen, "The Entity-Relationship Model – Toward a Unified View of Data," *ACM TODS*, 1(1), 1976.

[5] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *JMLR*, 12, 2011.

[6] T. Hastie, R. Tibshirani, J. Friedman, *The Elements of Statistical Learning*, 2nd ed., Springer, 2009.

[7] J. Brooke, "SUS: A quick and dirty usability scale," in *Usability Evaluation in Industry*, 1996.